

Proyecto 2 (%)

Fecha de Entrega:

Motor de Procesamiento Distribuido desde Cero

1 Objetivo de Aprendizaje

Diseñar e implementar un sistema distribuido desde cero que ejercite conceptos de **procesos/hilos, IPC/redes, planificación, memoria, sistema de archivos y coordinación distribuida**. El sistema debe exponer una **API cliente** para enviar trabajos y consultar estado, y ejecutar dichos trabajos en un **cluster** sencillo de nodos workers coordinados por un master.

2 Descripción General

Cada equipo implementará **una** de las dos rutas:

Ruta A: Batch DAG (mini-Spark): motor por *job* con **DAG** de etapas (map, filter, reduce, join, aggregate) sobre datos de archivos. Planifica tareas, maneja particiones y reintentos.

Ruta B: Streaming (mini-Flink): motor por *topología* con **operadores** (map, filter, keyed window aggregate) sobre flujos. Debe soportar ventanas por tiempo y *checkpointing* simplificado.

El proyecto es **académico** (no producción). Se prioriza claridad de diseño, cobertura de pruebas y evidencia de conceptos de SO.

3 Lenguaje, Entorno y Restricciones

- Lenguaje: **Rust** o **Go**.
- Prohibido usar frameworks de computación distribuida/streaming (Spark/Flink/Ray/Temporal/etc.). Permitidas librerías estándar de red (TCP/UDP/HTTP), serialización (JSON/MsgPack), *logging*, y utilidades básicas.
- Ejecución multi-proceso o multi-hilo por nodo. Comunicación inter-nodo por sockets TCP o HTTP/1.1.
- Despliegue local multinodo: múltiples procesos en la misma máquina o en varias máquinas (opcional). Se debe usar un `docker-compose` *simple*.
- Reproducibilidad: `Makefile` o scripts `bash` para build, test y demo.

4 Arquitectura Mínima Requerida

4.1 Componentes

1. Master/Coordinator:

- Registro de workers y *heartbeats*.
- Recepción de *jobs* (Batch) o *topologías* (Streaming).
- Planificador: asigna tareas/operadores a workers; política básica (round-robin + awareness de carga).
- Persistencia mínima del estado del job/topología (en archivos o `sqlite` local).

2. Workers:

- Ejecución de tareas/operadores aisladas en hilos o procesos.
- Manejo de **particiones** de datos (Batch) o **buffers** de eventos (Streaming).
- **Reintentos** y **reportes** de estado (éxito/falla, métricas).

3. Cliente (CLI):

- Envío de job/topología vía API.
- Consulta de estado, progreso, métricas y descarga de resultados.

4.2 Coordinación y Tolerancia a Fallos (Simulada)

- **Heartbeats** (cada 1–3 s) desde workers; si un worker deja de latir, el master lo marca *DOWN* y replanifica tareas pendientes.
- **Reintentos**: al menos 1 reintento por tarea fallida.
- **Checkpoints** (sólo Ruta B): snapshot periódico del estado de ventanas/keys a disco local (*best-effort*).
- **Idempotencia** básica: evitar duplicar resultados en reejecuciones (e.g., *task attempt id*).

4.3 Almacenamiento y Memoria

- **Batch**: cache en memoria por partición con *spill* a disco cuando supere umbral configurable.
- **Streaming**: colas/buffers con **backpressure** simple (bloqueo o caída controlada de tasa).

5 API (Especificación)

5.1 Contrato HTTP/JSON mínimo

- `POST /api/v1/jobs` (Batch): cuerpo JSON con dag (nodos, edges), inputs, paralelismo, partitions.
- `POST /api/v1/topologies` (Streaming): operadores y wiring; ventanas (tamaño, *slide*); claves.

- GET /api/v1/jobs/{id} o /topologies/{id}: estado (ACCEPTED/RUNNING/FAILED/SUCCEEDED), progreso (%), métricas.
- GET /api/v1/jobs/{id}/results: URL/paths de salida (Batch).
- POST /api/v1/ingest (Streaming): endpoint opcional para inyectar eventos (JSON por línea).

5.2 Formato de Job/Topología

```
{
  "name": "wordcount-batch",
  "dag": {
    "nodes": [
      {"id": "read", "op": "read_csv", "path": "data/*.csv", "partitions": 4},
      {"id": "flat", "op": "flat_map", "fn": "tokenize"},
      {"id": "map1", "op": "map", "fn": "to_lower"},
      {"id": "agg", "op": "reduce_by_key", "key": "token", "fn": "sum"}
    ],
    "edges": [["read", "flat"], ["flat", "map1"], ["map1", "agg"]]
  },
  "parallelism": 4
}
```

6 Operadores Mínimos por Ruta

Comunes

- map
- filter
- flat_map
- reduce / reduce_by_key

Ruta A (Batch DAG)

- **join** por clave entre dos colecciones.
- **shuffle** entre etapas (reparto de claves → particiones).
- Lectura/escritura: CSV y JSONL.

Ruta B (Streaming)

- **ventanas** por tiempo (tumbling y/o sliding).
- **keyed aggregate** con estado por clave.
- Ingesta por stdin, archivo *tail* o /api/v1/ingest.

7 Planificador y Ejecución

- **Planificación:** cola de tareas en el master + asignación *round-robin con carga*.
- **Ejecución:** cada worker mantiene un *pool* de hilos/procesos configurables.
- **Aislamiento:** cada tarea se ejecuta con límites de memoria/tiempo configurables (terminación si excede).

8 Métricas y Observabilidad

- Por nodo: uso de CPU (aprox.), memoria, número de tareas activas, latencia promedio, #reintentos.
- Por job/topología: tiempo total, etapas, throughput (Streaming), #fallos.
- *Logging* estructurado (JSON o texto) con niveles.

9 Casos de Prueba y Datasets

Batch recomendado

- **WordCount** en CSV/JSONL (text, ts).
- **Joins:** ventas & catálogo (100–500 MB totales con archivos repetidos para particiones).

Streaming recomendado

- Flujo de **logs** (JSONL con ts, level, service, msg) y agregación por ventana.

10 Hitos por Semana (4 Semanas)

1. **Semana 1 — Diseño y scaffolding:** documento de arquitectura (proceso master/worker, IPC, API), prototipo de registro de workers y healthcheck.
2. **Semana 2 — Planificación y ejecución básica:** envío de job/topología, ejecución de *map/filter*, métrica de progreso.
3. **Semana 3 — Operadores avanzados y tolerancia a fallos:** *reduce_by_key* + (join o ventanas), reintentos y replanificación.
4. **Semana 4 — Observabilidad, pulido y demo:** métricas, *logging*, Makefile, README, video y defensa.

11 Instrucciones de Desarrollo

- Estructurar el repositorio con /master, /worker, /client, /docs, /scripts.
- **Pruebas:** unitarias (operadores), integración (nodo único) y end-to-end (multinodo local).
- **Fallos simulados:** matar un worker durante la ejecución y demostrar recuperación.
- **Benchmarks mínimos:** lote de 1M registros (Batch) o 5k eventos/s durante 60s (Streaming), con reporte.

12 Restricciones y Buenas Prácticas

- Sin frameworks distribuidos; escribir su propio planificador y ejecución.
- Serialización consistente (JSON o MsgPack) y **versionado** del mensaje.
- No bloquear el master con trabajo pesado (sólo coordinación).
- Manejo de señales para apagado ordenado.

13 Entregables

- Código fuente en repo con README (build/ejecución), Makefile, scripts.
- **Documento de arquitectura:** modelo de procesos/hilos, API, protocolos, planificación, memoria, fallos.
- **Suite de pruebas** (con cómo ejecutarlas).
- **Video demostrativo:** instalación, ejecución, casos de prueba (incl. fallo simulado).
- **Reporte de benchmarks:** entorno, parámetros y resultados.

14 Política de Entrega

- La entrega se debe realizar antes de las 10:00 pm del día de la entrega en un archivo .zip mediante el TecDigital, en los grupos de trabajo previamente establecidos.
- La entrega tardía se penaliza con 5 puntos porcentuales por cada 24 horas de retraso acumuladas.

15 Rúbrica de Evaluación

Criterio	Pond.	Descripción
Diseño y documento de arquitectura	15%	Claridad, decisiones de SO, diagramas.
API y cliente (contrato y usabilidad)	10%	Endpoints, validación, CLI usable.
Coordinación y planificación	15%	Registro, heartbeats, asignación, reintentos.
Ejecución distribuida y operadores mínimos	20%	map/filter/flat_map + reduce_by_key; (join o ventanas).
Tolerancia a fallos (simulada)	10%	Detección, replanificación, idempotencia básica.
Memoria/Almacenamiento & backpressure	10%	Cache/Spill (Batch) o buffers/ventanas (Streaming).
Pruebas (unitarias/integración/E2E)	10%	Cobertura y automatización.
Observabilidad y métricas	5%	Logs, métricas por nodo y por job/topología.
Demo y benchmarks	5%	Video claro, escenarios y reporte de rendimiento.
Calidad del código y repositorio	5%	Organización, lectura, scripts de build.
Total	100%	

16 Notas y Alcances

- Se evaluará el **entendimiento de SO** evidenciado en el diseño y en la implementación, no sólo la funcionalidad.
- Se permite usar IA como apoyo (búsqueda, documentación); cualquier uso debe declararse en el README.